Functions in Python – Complete Detailed Notes

1. What is a Function?

A **function** is a reusable block of code designed to perform a specific task.

Instead of writing the same code again and again, we define a function once and call it whenever needed.

Functions help in:

- Reducing repetition
- Organizing code
- Improving readability
- Reusing logic
- Easier debugging/testing

2. Why Use Functions?

Without functions:

```
print("Welcome")
print("Welcome")
print("Welcome")
```

With functions:

```
def greet():
    print("Welcome")

greet()
greet()
greet()
```

3. Syntax of Function

```
def function_name():
    # code block
```

Example:

```
def hello():
    print("Hello Python")
```

4. Calling a Function

A function runs only when called.

```
def hello():
    print("Hello")

hello()
```

Output:

Hello

5. Parts of a Function

```
def add():
    print(2 + 3)
```

- def → keyword to define function
- add → function name
- () → parentheses

- : → starts block
- indented code → body

6. Function with Parameters

Parameters are inputs given to function.

```
def greet(name):
    print("Hello", name)

greet("Aadipal")
```

Output:

Hello Aadipal

7. Multiple Parameters

```
def add(a, b):
    print(a + b)

add(5, 3)
```

8. Arguments vs Parameters

Term	Meaning
Parameter	Variable in function definition
Argument	Value passed during call

```
def show(name):   # parameter
    print(name)
```

```
show("Ram")      # argument
```

9. Return Statement

Used to send result back.

```
def add(a, b):
   return a + b

result = add(2, 3)
print(result)
```

Output:

5

10. Difference: print() vs return

print()	return
Displays output	Sends value back
Cannot reuse directly	Can store/use later

Example:

```
def square(n):
   return n * n
```

11. Default Parameters

If no value passed, default used.

```
def greet(name="Guest"):
    print("Hello", name)

greet()
greet("Ravi")
```

Output:

```
Hello Guest
Hello Ravi
```

12. Keyword Arguments

Pass values by parameter name.

```
def student(name, age):
    print(name, age)

student(age=20, name="Aman")
```

13. Positional Arguments

Values passed by order.

```
student("Aman", 20)
```

14. Arbitrary Arguments *args

Accepts multiple values.

```
def total(*nums):
    print(sum(nums))
```

```
total(1,2,3,4)
```

15. Keyword Arbitrary Arguments **kwargs

Stores named values.

```
def info(**data):
    print(data)

info(name="Ravi", age=21)
```

Output:

```
{'name': 'Ravi', 'age': 21}
```

16. Types of Functions in Python

A. Built-in Functions

Already available in Python.

Examples:

- print()
- len()
- sum()
- type()
- max()

B. User-defined Functions

Created using def

C. Lambda Functions

Anonymous one-line functions

17. Lambda Function

```
square = lambda x: x * x
print(square(5))
```

Output:

25

18. Recursive Function

A function calling itself.

```
def countdown(n):
    if n == 0:
        return
    print(n)
    countdown(n-1)

countdown(5)
```

19. Factorial Using Recursion

```
def fact(n):
    if n == 1:
        return 1
    return n * fact(n-1)

print(fact(5))
```

Output:

20. Local and Global Variables

Local Variable

Inside function only.

```
def show():
    x = 10
```

Global Variable

Outside function.

```
x = 100

def show():
    print(x)
```

21. Using global Keyword

```
x = 5

def change():
    global x
    x = 10

change()
print(x)
```

Output:

```
10
```

22. Nested Functions

Function inside another function.

```
def outer():
    def inner():
        print("Inner")
    inner()

outer()
```

23. Function Returning Multiple Values

```
def calc(a, b):
    return a+b, a-b

x, y = calc(5, 2)
print(x, y)
```

24. Docstring in Function

Description of function.

```
def add(a, b):
    """Returns sum of two numbers"""
    return a+b
```

25. Built-in Useful Functions

Function	Work
len()	Length

max()	Largest
min()	Smallest
sum()	Total
sorted()	Sort values
type()	Data type

26. Scope of Variables

LEGB Rule:

- **L** → Local
- **E** → Enclosed
- **G** → Global
- **B** → Built-in

Python checks variables in this order.

27. First-Class Functions

Functions can be:

- stored in variables
- passed as arguments
- returned from other functions

```
def greet():
  print("Hi")

x = greet
x()
```

28. Higher Order Functions

Function takes another function.

```
def apply(func, value):
    return func(value)

print(apply(abs, -5))
```

29. Map, Filter, Reduce

map()

```
nums = [1,2,3]
print(list(map(lambda x:x*2, nums)))
```

filter()

```
nums = [1,2,3,4]
print(list(filter(lambda x:x%2==0, nums)))
```

30. Best Practices

✅ Use meaningful names

✅ Keep functions small

✅ Use return when needed

✅ Reuse code

✅ Add docstrings

❌ Avoid:

- Very long functions
- Too many parameters
- Repeated code

31. Real Life Uses

- Calculator operations
- Login system
- File handling tasks
- Data processing
- Web applications
- Game logic

32. Short Notes for Exam

- def defines function
- Parameters = inputs
- return gives output
- Lambda = one-line function
- Recursion = function calls itself
- Global variable = outside function

33. Viva Questions

Q1. Difference between parameter and argument?

Parameter = variable in definition

Argument = actual value passed

Q2. What is recursion?

Function calling itself.

Q3. What is lambda?

Anonymous one-line function.

Q4. Why use functions?

Reuse code and organize program.

34. Memory Trick

D C P R

- **D**efine
- **C**all
- **P**arameters
- **R**eturn

35. One-Line Summary

Functions in Python are reusable blocks of code used to perform specific tasks efficiently and organize programs better.